

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ
по лабораторной работе №3
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Динамические списки»

Выполнили:

студенты группы 24ВВВ3

Алмакаев В.В.

Буров А.О.

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н., Деев М.В.

Пенза 2025

Цель работы – Освоить принципы работы с динамическими структурами данных на примере односвязных списков, включая их создание, манипулирование элементами и организацию памяти.

Лабораторное задание:

Задание

1. Реализовать приоритетную очередь, путём добавления элемента в список в соответствии с приоритетом объекта (т.е. объект с большим приоритетом становится перед объектом с меньшим приоритетом).
2. * На основе приведенного кода реализуйте структуру данных *Очередь*.
3. * На основе приведенного кода реализуйте структуру данных *Стек*.

Задание 1.

Код программы

C#

using System;

public class Node

{

public string inf;

public int priority; // Только для приоритетной очереди

public Node? next;

public Node(string data, int prio = 0) // prio теперь необязательный

{

inf = data;

priority = prio;

next = null;

}

}

public class PriorityQueue

```

{

    private Node? head = null;

    private Node? last = null;

    private int dlinna = 0;


    // ОТДЕЛЬНЫЙ метод для создания элемента приоритетной очереди

    private Node? GetPriorityStruct()
    {

        Console.Write("Введите название объекта: ");

        string? s = Console.ReadLine();


        if (string.IsNullOrEmpty(s))
        {

            Console.WriteLine("Запись не была произведена");

            return null;

        }


        Console.Write("Введите приоритет объекта (чем БОЛЬШЕ - тем ВЫШЕ): ");

        string? priorityInput = Console.ReadLine();


        if (string.IsNullOrEmpty(priorityInput) || !int.TryParse(priorityInput, out int
priority))
        {

            Console.WriteLine("Ошибка ввода приоритета");

            return null;

        }


        return new Node(s, priority);

    }
}

```

// ОТДЕЛЬНЫЙ метод для создания простого элемента (для FIFO и стека)

private Node? GetSimpleStruct()

{

Console.Write("Введите название объекта: ");

string? s = Console.ReadLine();

if (string.IsNullOrEmpty(s))

{

Console.WriteLine("Запись не была произведена");

return null;

}

return new Node(s); // Без приоритета

}

// ЗАДАНИЕ 1: Приоритетная очередь (чем БОЛЬШЕ число - тем ВЫШЕ приоритет)

public void AddToPriorityQueue()

{

Node? p = GetPriorityStruct();

if (p == null) return;

if (head == null)

{

head = p;

last = p;

}

else

```
{  
  
    Node? current = head;  
  
    Node? prev = null;  
  
    // ИЗМЕНИЛ УСЛОВИЕ: теперь ищем элемент с МЕНЬШИМ приоритетом  
    while (current != null && current.priority >= p.priority)  
    {  
        prev = current;  
        current = current.next;  
    }  
  
    if (prev == null)  
    {  
        // Вставляем в начало (новый самый приоритетный)  
        p.next = head;  
        head = p;  
    }  
    else  
    {  
        // Вставляем в середину или конец  
        prev.next = p;  
        p.next = current;  
  
        if (current == null)  
        {  
            last = p;  
        }  
    }  
}
```

```

    }

    dlinna++;

    Console.WriteLine($"Добавлен в приоритетную очередь: '{p.inf}' с приоритетом
    {p.priority}");
}

// Просмотр ТОЛЬКО приоритетной очереди
public void ReviewPriorityQueue()
{
    Node? current = head;

    if (current == null)
    {
        Console.WriteLine("Приоритетная очередь пуста");

        return;
    }

    Console.WriteLine("\n--- Приоритетная очередь ---");

    int position = 1;

    while (current != null)
    {
        Console.WriteLine($"{position}.        Имя:        {current.inf},        Приоритет:
        {current.priority}");

        current = current.next;

        position++;
    }

    Console.WriteLine($"Всего элементов: {dlinna}");
}

// Поиск в приоритетной очереди

```

```

public Node? FindInPriorityQueue(string name)
{
    Node? current = head;
    while (current != null)
    {
        if (current.inf == name)
        {
            return current;
        }
        current = current.next;
    }
    Console.WriteLine($"Элемент '{name}' не найден в приоритетной очереди");
    return null;
}

```

// Удаление из приоритетной очереди

```

public void DeleteFromPriorityQueue(string name)
{
    if (head == null)
    {

        Console.WriteLine("Приоритетная очередь пуста");
        return;
    }

    if (head.inf == name)
    {

```

```
Console.WriteLine($"Удален из приоритетной очереди: '{head.inf}'");

head = head.next;

dlinna--;

if (head == null) last = null;

return;
}

Node? current = head;

Node? prev = null;

while (current != null && current.inf != name)
{
    prev = current;
    current = current.next;
}

if (current == null)
{
    Console.WriteLine($"Элемент '{name}' не найден в приоритетной очереди");
    return;
}

Console.WriteLine($"Удален из приоритетной очереди: '{current.inf}'");

if (prev != null)
{
    prev.next = current.next;
}
```



```

        if (current == last)
        {
            last = prev;
        }

        dlinna--;
    }

    // Извлечение из приоритетной очереди (самый приоритетный)
    public void DequeueFromPriorityQueue()
    {
        if (head == null)
        {
            Console.WriteLine("Приоритетная очередь пуста");
            return;
        }

        Console.WriteLine($"Извлечен из приоритетной очереди: '{head.inf}' (приоритет: {head.priority})");
        head = head.next;
        dlinna--;

        if (head == null)
        {
            last = null;
        }
    }
}

```

```
public class SimpleQueue
```

```
{
```

```
    private Node? head = null;
```

```
    private Node? last = null;
```

```
    private int dlinna = 0;
```

```
    // ЗАДАНИЕ 2: Очередь FIFO
```

```
    public void Enqueue(string data)
```

```
    {
```

```
        Node p = new Node(data); // Создаем без приоритета
```

```
        if (head == null)
```

```
        {
```

```
            head = p;
```

```
            last = p;
```

```
        }
```

```
        else
```

```
        {
```

```
            if (last != null)
```

```
            {
```

```
                last.next = p;
```

```
            }
```

```
            last = p;
```

```
        }
```

```
        dlinna++;
```

```
        Console.WriteLine($"Добавлен в очередь FIFO: '{p.inf}');"
```

```
    }
```

```
// Отдельный метод для ввода данных и добавления в FIFO

public void AddToFifoQueue()
{
    Console.Write("Введите название объекта для очереди FIFO: ");
    string? s = Console.ReadLine();

    if (string.IsNullOrEmpty(s))
    {
        Console.WriteLine("Запись не была произведена");
        return;
    }

    Enqueue(s);
}

// Просмотр ТОЛЬКО очереди FIFO

public void ReviewFifoQueue()
{
    Node? current = head;
    if (current == null)
    {
        Console.WriteLine("Очередь FIFO пуста");
        return;
    }

    Console.WriteLine("\n--- Очередь FIFO ---");
    int position = 1;
    while (current != null)
```

```

    {
        Console.WriteLine($"{position}. Имя: {current.inf}");
        current = current.next;
        position++;
    }
    Console.WriteLine($"Всего элементов: {dlinna}");
}

// Извлечение из очереди FIFO
public void DequeueFromFifo()
{
    if (head == null)
    {
        Console.WriteLine("Очередь FIFO пуста");
        return;
    }

    Console.WriteLine($"Извлечен из очереди FIFO: '{head.inf}'");
    head = head.next;
    dlinna--;

    if (head == null)
    {
        last = null;
    }
}
}

```

```

public class Stack
{
    private Node? head = null;

    private Node? last = null;

    private int dlinna = 0;

    // ЗАДАНИЕ 3: Стек LIFO

    public void Push()
    {
        Console.Write("Введите название объекта для стека: ");

        string? s = Console.ReadLine();

        if (string.IsNullOrEmpty(s))
        {
            Console.WriteLine("Запись не была произведена");

            return;
        }

        Node p = new Node(s); // Без приоритета

        if (head == null)
        {
            head = p;

            last = p;
        }
        else
        {
            if (last != null)

```

```
    {  
        last.next = p;  
    }  
    last = p;  
}  
dlinna++;  
Console.WriteLine($"Добавлен в стек: '{p.inf}');  
}
```

// Просмотр ТОЛЬКО стека

public void ReviewStack()

```
{  
    Node? current = head;  
    if (current == null)  
    {  
        Console.WriteLine("Стек пуст");  
        return;  
    }  
}
```

Console.WriteLine("\n--- Стек LIFO ---");

int position = 1;

while (current != null)

```
{  
    Console.WriteLine($"{position}. Имя: {current.inf}");  
    current = current.next;  
    position++;  
}
```

```

    }

    Console.WriteLine($"Всего элементов: {dlinna}");
}

// Извлечение из стека LIFO

public void PopFromStack()
{
    if (head == null)
    {
        Console.WriteLine("Стек пуст");
        return;
    }

    if (head == last)
    {
        Console.WriteLine($"Извлечен из стека: '{head.inf}'");
        head = null;
        last = null;
        dlinna--;
        return;
    }

    Node? current = head;

    while (current != null && current.next != last)
    {
        current = current.next;
    }

```

```

        if (current == null || last == null) return;

        Console.WriteLine($"Извлечен из стека: '{last.inf}'");

        last = current;

        last.next = null;

        dlinna--;

    }
}

class Program
{
    static void Main()
    {
        PriorityQueue priorityQueue = new PriorityQueue();

        SimpleQueue fifoQueue = new SimpleQueue();

        Stack stack = new Stack();

        Console.WriteLine("=== Управление структурами данных ===");

        Console.WriteLine("Приоритетная очередь: чем БОЛЬШЕ число - тем ВЫШЕ приоритет\n");

        while (true)
        {
            Console.WriteLine("\n=== ГЛАВНОЕ МЕНЮ ===");

            Console.WriteLine("=== ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ===");

            Console.WriteLine("1. Добавить элемент в приоритетную очередь");

            Console.WriteLine("2. Просмотреть приоритетную очередь");

            Console.WriteLine("3. Найти элемент в приоритетной очереди");

            Console.WriteLine("4. Удалить элемент из приоритетной очереди");

```



```
Console.WriteLine("5. Извлечь из приоритетной очереди (самый приоритетный)");
```

```
Console.WriteLine("\n=== ОЧЕРЕДЬ FIFO ===");
```

```
Console.WriteLine("6. Добавить элемент в очередь FIFO");
```

```
Console.WriteLine("7. Просмотреть очередь FIFO");
```

```
Console.WriteLine("8. Извлечь из очереди FIFO (первый пришел)");
```

```
Console.WriteLine("\n=== СТЕК LIFO ===");
```

```
Console.WriteLine("9. Добавить элемент в стек");
```

```
Console.WriteLine("10. Просмотреть стек");
```

```
Console.WriteLine("11. Извлечь из стека (последний пришел)");
```

```
Console.WriteLine("\n12. Выход");
```

```
Console.Write("Выберите действие: ");
```

```
string? input = Console.ReadLine();
```

```
if (string.IsNullOrEmpty(input) || !int.TryParse(input, out int choice))
```

```
{
```

```
    Console.WriteLine("Неверный ввод!");
```

```
    continue;
```

```
}
```

```
switch (choice)
```

```
{
```

```
    // Приоритетная очередь
```

```
    case 1:
```

```
        priorityQueue.AddToPriorityQueue();
```

```
        break;
```

case 2:

```
priorityQueue.ReviewPriorityQueue();  
break;
```

case 3:

```
Console.Write("Введите имя для поиска в приоритетной очереди: ");  
string? findNamePriority = Console.ReadLine();  
if (!string.IsNullOrEmpty(findNamePriority))  
{  
    Node? found = priorityQueue.FindInPriorityQueue(findNamePriority);  
    if (found != null)  
    {  
        Console.WriteLine($"Найден: '{found.inf}' с приоритетом {  
found.priority} ");  
    }  
}  
break;
```

case 4:

```
Console.Write("Введите имя для удаления из приоритетной очереди: ");  
string? delNamePriority = Console.ReadLine();  
if (!string.IsNullOrEmpty(delNamePriority))  
{  
    priorityQueue.DeleteFromPriorityQueue(delNamePriority);  
}  
break;
```

case 5:

```
priorityQueue.DequeueFromPriorityQueue();  
break;
```

// Очередь FIFO

case 6:

fifoQueue.AddToFifoQueue();

break;

case 7:

fifoQueue.ReviewFifoQueue();

break;

case 8:

fifoQueue.DequeueFromFifo();

break;

// Стек LIFO

case 9:

stack.Push();

break;

case 10:

stack.ReviewStack();

break;

case 11:

stack.PopFromStack();

break;

case 12:

Console.WriteLine("Выход из программы...");

return;

default:

Console.WriteLine("Неверный выбор!");

break;

}

```
}  
  
}  
  
}
```

Результат работы программы

```
=== Управление структурами данных ===  
Приоритетная очередь: чем БОЛЬШЕ число - тем ВЫШЕ приоритет  
  
=== ГЛАВНОЕ МЕНЮ ===  
=== ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ===  
1. Добавить элемент в приоритетную очередь  
2. Просмотреть приоритетную очередь  
3. Найти элемент в приоритетной очереди  
4. Удалить элемент из приоритетной очереди  
5. Извлечь из приоритетной очереди (самый приоритетный)  
  
=== ОЧЕРЕДЬ FIFO ===  
6. Добавить элемент в очередь FIFO  
7. Просмотреть очередь FIFO  
8. Извлечь из очереди FIFO (первый пришел)  
  
=== СТЕК LIFO ===  
9. Добавить элемент в стек  
10. Просмотреть стек  
11. Извлечь из стека (последний пришел)  
  
12. Выход  
Выберите действие:
```

Рисунок 1 - Результат работы программы 1

Задание 3.

Результат работы программы

```
Выберите действие: 2  
  
--- Приоритетная очередь ---  
1. Имя: o, Приоритет: 12  
2. Имя: h, Приоритет: 3  
3. Имя: g, Приоритет: 1  
Всего элементов: 3  
  
=== ГЛАВНОЕ МЕНЮ ===  
=== ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ===  
1. Добавить элемент в приоритетную очередь  
2. Просмотреть приоритетную очередь  
3. Найти элемент в приоритетной очереди  
4. Удалить элемент из приоритетной очереди  
5. Извлечь из приоритетной очереди (самый приоритетный)  
  
=== ОЧЕРЕДЬ FIFO ===  
6. Добавить элемент в очередь FIFO  
7. Просмотреть очередь FIFO  
8. Извлечь из очереди FIFO (первый пришел)  
  
=== СТЕК LIFO ===  
9. Добавить элемент в стек  
10. Просмотреть стек  
11. Извлечь из стека (последний пришел)  
  
12. Выход
```

Рисунок 2 - Результат работы программы 2

Задание 3.

Результат работы программы

```
Выберите действие: 11
Извлечен из стека: 'lol13'

=== ГЛАВНОЕ МЕНЮ ===
=== ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ===
1. Добавить элемент в приоритетную очередь
2. Просмотреть приоритетную очередь
3. Найти элемент в приоритетной очереди
4. Удалить элемент из приоритетной очереди
5. Извлечь из приоритетной очереди (самый приоритетный)

=== ОЧЕРЕДЬ FIFO ===
6. Добавить элемент в очередь FIFO
7. Просмотреть очередь FIFO
8. Извлечь из очереди FIFO (первый пришел)

=== СТЕК LIFO ===
9. Добавить элемент в стек
10. Просмотреть стек
11. Извлечь из стека (последний пришел)

12. Выход
Выберите действие: 10

--- Стек LIFO ---
1. Имя: lol1
2. Имя: lol2
Всего элементов: 2
```

Рисунок 3 - Результат работы программы 3

Вывод: В ходе выполнения лабораторной работы были успешно освоены принципы работы с динамическими структурами данных на примере односвязных списков.